

## 14. 딥러닝 맛보기 - TensorFlow

파이썬 외부 라이브러리

인공 신경망, 기계학습과 딥러닝

TensorFlow, Keras, 구글 colab 이용

이미지 분류 학습 예제-TensorFlow와 Keras를 이용

출처: 따라하며 배우는 파이썬과 데이터 과학. 생능출판사  
그림과 함께하는 파이썬 수업. 생능출판사

# 파이썬 외부 라이브러리

---

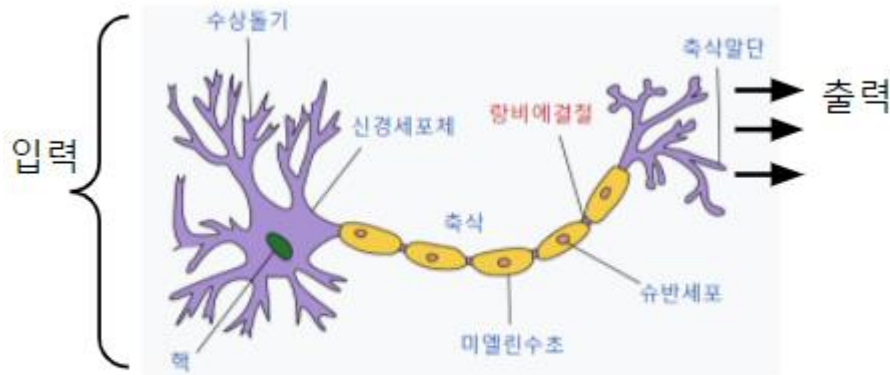
- 기계학습 (또는 딥러닝) 라이브러리



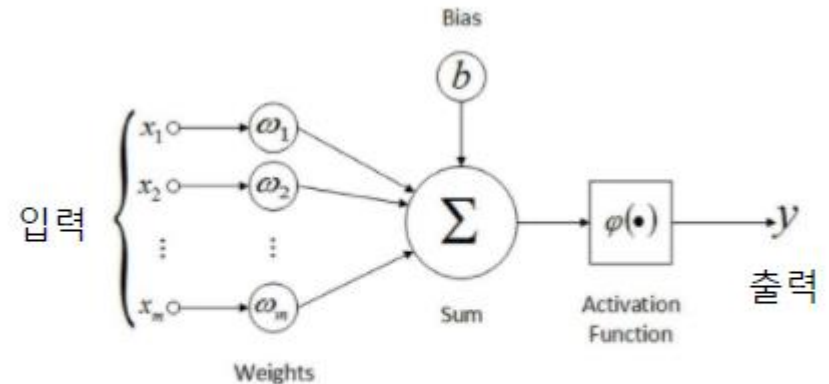
- 텐서플로우Tensorflow는 머신러닝과 딥러닝을 위한 오픈소스 플랫폼.
- 텐서플로우는 우수한 기능과 서비스를 제공, 병렬처리를 지원, 고급 신경망 네트워크 모델을 쉽게 구현할 수 있게 해줌

# 인공 신경망

- 인간의 뇌는 수없이 많은 신경세포가 다른 신경세포와 연결되어 있다
- 신경세포속 수상돌기는 다른 신경세포로부터 신호를 전달받아서(입력) 이 신호에 대한 반응을 다른 신경세포로 전달한다(출력)
- 인공적인 신경세포artificial neuron를 흉내내는 프로그램을 퍼셉트론perceptron이라고 한다.



인간의 신경세포

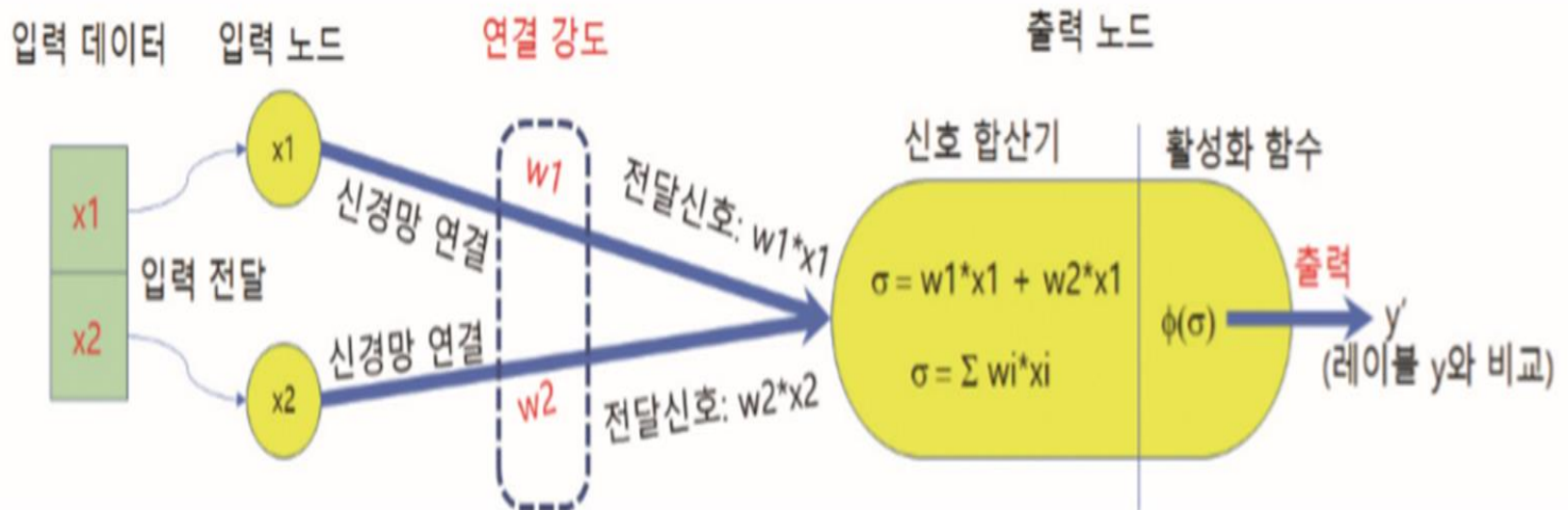


인공적인 신경세포

# 인공 신경망

- 가장 간단한 인공 신경망

- 입력 데이터는 입력 노드에 전달, 입력 노드는 출력 노드로 연결되어 있다. 이 신경망 연결을 통해 신호가 전달되는 것.
- 출력 노드는 두 개의 기능으로 구분할 수 있다.
  - 전달되어 오는 신호를 모두 합하는 신호 합산기.
  - 합산된 신호에 따라 다음 계층으로 신호를 보낼지 말지는 결정하는 활성화(activation) 함수.



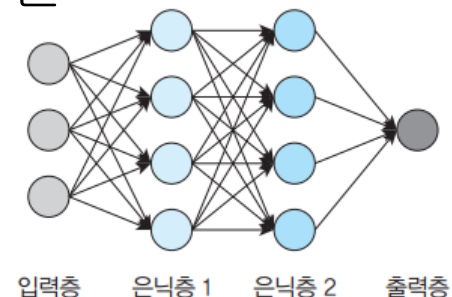
# 기계학습과 딥러닝

- 기계학습

- 많은 데이터(빅데이터)가 축적되면서 기계가 스스로 학습할 수 있는 방법론
- 인공신경망(ANN: Artificial Neural Network)이 가장 효과적인 기법
  - 인간의 신경세포(Neuron)를 모방한 컴퓨터 모델
  - 문자 인식, 이미지 인식, 음성 인식과 같은 영역에서 좋은 결과

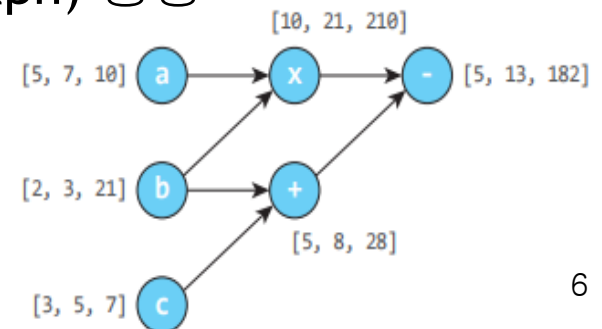
- 딥러닝(Deep Learning) 모델

- 2012년 토론토 대학의 제프리 힌튼(Jeffrey Hinton) 우수성 입증
- 요즘 대부분의 인공신경망은 딥러닝 모델을 채택
- 인공신경망에서 입력과 출력 사이에 존재하는 은닉층(Hidden Layer)의 층수가 큰 경우



# TensorFlow

- 가장 인기있는 기계학습(machine learning)과 딥러닝 플랫폼
- 구글 브레인 팀이 개발한 기계학습 지원 라이브러리
  - 구글사 내 인공지능 연구에 이용, 오픈소스 소프트웨어
  - 다양한 기계학습 기법 및 딥러닝 모델과 **Python API** 제공
- 딥러닝을 위한 **CNN, RNN** 및 오토인코더 기능도 지원
  - 숫자 인식, 이미지 인식에 적합한 합성 신경망  
(**CNN: Convolutional Neural Network**)
  - 자연어처리, 기계번역 등 시퀀스-투-시퀀스 순환신경망  
(**RNN: Recurrent Neural Network**)
- 이용자는 데이터 플로우 그래프(Dataflow Graph) 생성
  - 각 노드(Node)는 수리적 연산을 의미
  - 엣지(Edge)는 다차원 배열의 흐름을 의미
  - 이 다차원 배열을 텐서(Tensor)라 칭함



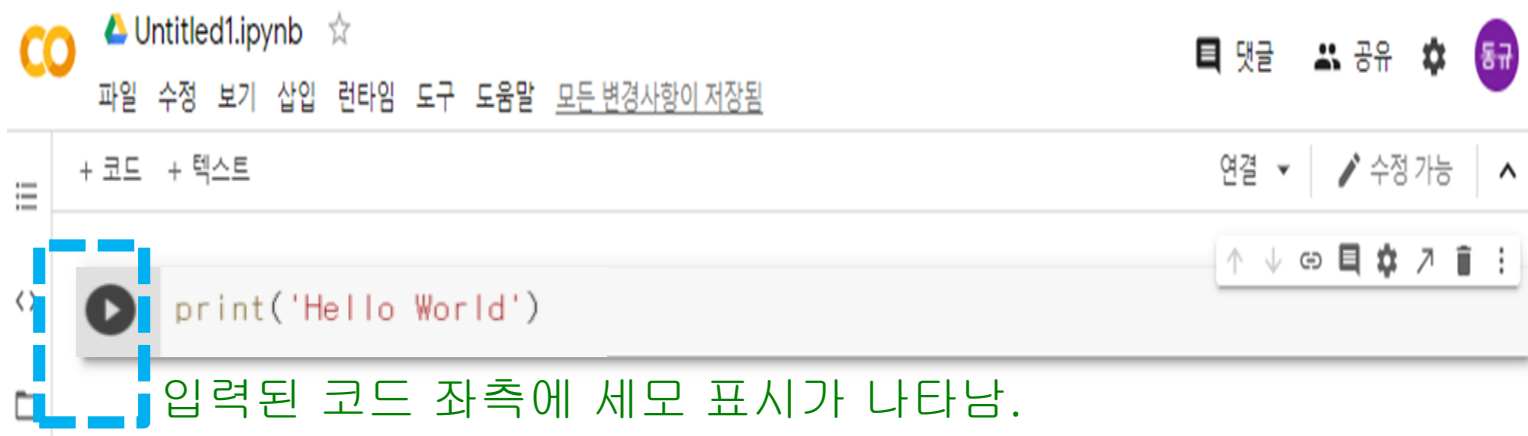
# Keras

---

- Keras는 TensorFlow 위에서 작동
  - TensorFlow가 기계학습을 위한 저수준의 기능을 지원, Keras는 고수준의 기능을 제공
  - 기계학습을 위한 분류(Classification), 회귀(Regression), 과대적합(Overfitting)/과소적합(Underfitting) 기능을 지원
  - 딥러닝을 위해 CNN 및 RNN을 지원
  - 딥러닝 프로토타입을 용이하고 빠르게 개발하고 싶은 경우 Keras를 이용하면 매우 편리

# 구글 colab 이용

- 구글의 **Colab**은 클라우드 환경에서 주피터 노트북 기반의 파이썬 개발을 위한 환경을 제공.
- **colab** 환경에서 실행하면
  - 코드 단위별 실행, 결과 이미지 바로 보기
  - 설치과정 없이 텐서플로우를 바로 이용할 수 있다.
- 코랩 서비스에 로그인을 하고
  - 새로운 파이썬 파일인 파이썬 노트북을 생성하고
  - 다음과 같이 **Hello World**를 출력하는 코드를 작성해 보자.



입력된 코드 좌측에 세모 표시가 나타남.  
이것을 선택하면 파이썬 코드가 번역되고 웹 환경에서 실행됨.  
실행 결과는 아래쪽에 나타난다. 이 셀 아래에 새로운 셀을 만들고  
명령어를 입력하여 실행시킬 수 있다.

# 구글 colab 이용

- 파이썬 코딩을 위해서는 코랩 환경 최상단의 '+ 코드' 메뉴를 선택. 이 메뉴를 선택할 때마다 코드를 입력할 수 있는 셀cell이 나타난다.
- 코드를 입력하면 좌측에 세모 표시가 나타남. 이 부분을 선택하면, 파이썬 코드가 번역되고 웹 환경에서 실행된다.
- 실행 결과는 실행 셀 아래쪽에 나타나며, 이 셀 아래에 새로운 셀을 만들고 명령어를 입력하여 실행시킬 수 있다.

실행되는 순서

```
[1] 1 print('Hello World')
```

☞ Hello World

실행결과

```
[2] 1 print('새로운 셀에서 파이썬을 실행합니다.')
```

☞ 새로운 셀에서 파이썬을 실행합니다.

```
[3] 1 import numpy as np      # 넘파이를 사용해 봅시다
    2 print(np.random.randn(10))
```

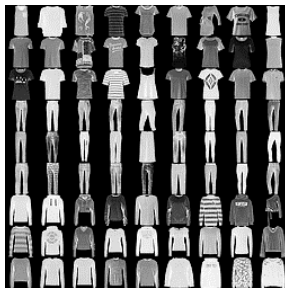
넘파이를 import 하기  
위한 별도의 설치과정  
이 필요없음

☞ [ 0.73717765 0.84198299 -0.23773511 -0.37813154 0.32783317 -1.11242014  
-0.28947043 0.72525395 0.27578142 0.18384656]

최상단의 '+ 코드' 명령으로  
코드 셀을 생성한다

# 이미지 분류 학습 예제: TensorFlow와 Keras를 이용

- 텐서플로우를 이용하여 다층 신경망을 구축해서 테스트해 본다.
  - 구축된 다층신경망에서 이미지를 구분하는 기능을 구현.
  - 신경망을 쌓아 가는 딥러닝 개념을 사용
- 인공지능 신경망 훈련 예제 : 패션 이미지 분류 프로그램
  - TensorFlow 커뮤니티에서 공개한 기본 튜토리얼
  - 셔츠, 바지, 가방, 구두 등 10가지의 패션 의류를 분류 학습하고 샘플 이미지로 평가
- 학습 데이터는 MIT에서 제작한 데이터셋 Fashion MNIST 사용
  - 28x28 픽셀 크기의 흑백 이미지 7000개가 10개의 클래스로 분류



...



# 예시 데이터: 패션 MNIST

- 패션 MNIST 데이터

- 다음과 같이 운동화나 셔츠같은 옷과 신발의 이미지와 이 이미지에 대한 레이블을 제공하고 있다.
- keras의 데이터셋에 있는데 이를 읽어와서 사용한다.



# 1) 환경 설정 및 데이터셋 로드

- colab 환경에서 실행: 코드 단위별 실행, 결과 이미지 바로 볼 수 있다

▶ 실행버튼

```
import tensorflow as tf    # tensorflow와 tf.keras를 임포트한다
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

# load\_data() 함수로 데이터셋 로드

# 패션 MNIST 데이터는 keras의 데이터셋에 있는데 이를 읽어와서

# 학습용, 테스트 데이터로 구분

```
fashion_mnist = keras.datasets.fashion_mnist
```

```
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```



```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
32768/29515 [=====] - 0s 0us/step
40960/29515 [=====] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26427392/26421880 [=====] - 0s 0us/step
26435584/26421880 [=====] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
16384/5148 [=====] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4423680/4422102 [=====] - 0s 0us/step
4431872/4422102 [=====] - 0s 0us/step
```

# 패션 MNIST는 레이블이 있는 이미지 데이터

- 읽어들인 데이터의 형태는 다음과 같이 확인



```
print(train_images.shape) # 학습 이미지의 형태와 레이블을 출력한다  
print(train_labels)  
print(test_images.shape)
```



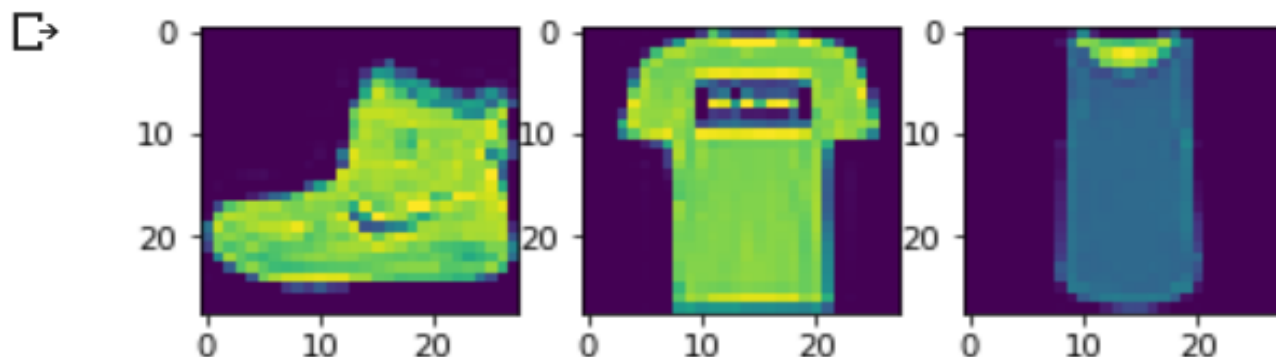
```
(60000, 28, 28)  
[9 0 0 ... 3 0 5]  
(10000, 28, 28)
```

# 패션 MNIST는 레이블이 있는 이미지 데이터

- 이미지 파일 형태인 학습데이터의 첫번째·두번째·세번째 데이터를 가져와서 각각 구두, 티셔츠, 티셔츠임을 확인.

```
▶ fig = plt.figure()
  ax1 = fig.add_subplot(1, 3, 1)
  ax2 = fig.add_subplot(1, 3, 2)
  ax3 = fig.add_subplot(1, 3, 3)

  ax1.imshow(train_images[0])      # 첫 번째 훈련용 데이터
  ax2.imshow(train_images[1])      # 두 번째 훈련용 데이터
  ax3.imshow(train_images[2])      # 세 번째 훈련용 데이터
  plt.show()
```



# 패션 MNIST는 레이블이 있는 이미지 데이터

- 이제 이 세 이미지의 레이블을 출력해 보도록 하자.
  - 이미지 레이블에 대한 상세한 서술은 데이터 내에는 없으며 케라스 홈페이지에서 얻을 수 있다.

```
print(train_labels[:3])
```

```
[9 0 0]
```



| 레이블 | 0       | 1       | 2        | 3     | 4    | 5      | 6     | 7       | 8   | 9          |
|-----|---------|---------|----------|-------|------|--------|-------|---------|-----|------------|
| 클래스 | T-shirt | Trouser | Pullover | Dress | Coat | Sandal | Shirt | Sneaker | Bag | Ankle boot |

# 딥러닝? 인공 신경망을 구축해 보자

- 딥러닝(**deep learning**)은 인공신경망의 층(**layer**)을 깊이 쌓아 학습을 하겠다는 것.
- 다음과 같은 코드로 몇 개의 신경망 층을 형성해 보자.

```
▶ model = keras.Sequential([  
    keras.layers.Flatten(input_shape=(28, 28)),  
    keras.layers.Dense(128, activation='relu'),  
    keras.layers.Dense(10, activation='softmax')  
])
```

**#이것은 다음과 같은 계층 구조를 의미**

3개의 층을 가진 네트워크 모델을 생성한다

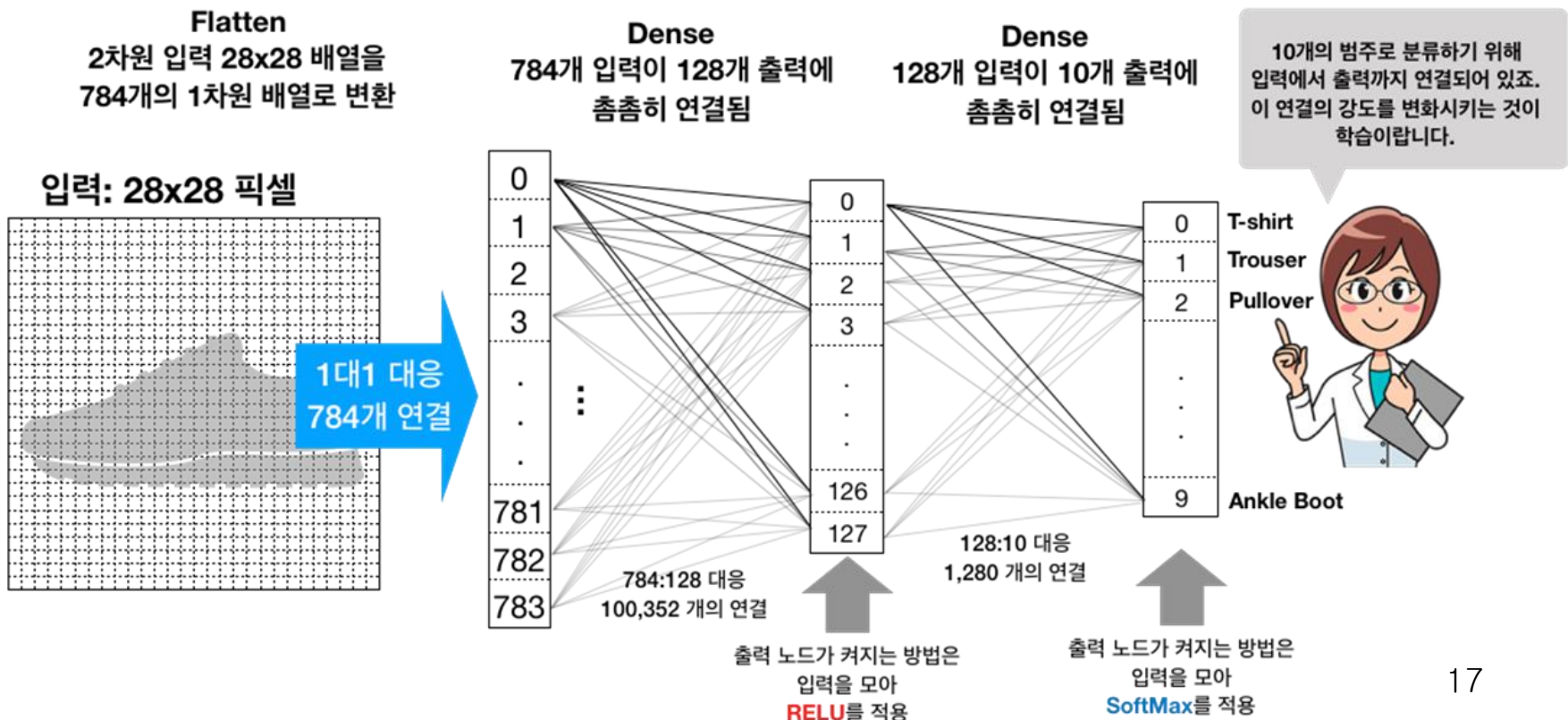
1. Flatten 계층: 입력은 (28, 28)

2. Dense 네트워크: 출력 128, 활성화 방법: **RELU**

3. Dense 네트워크: 출력 10, 활성화 방법: **SoftMax**

# 딥러닝? 인공 신경망을 구축해 보자

- 1차원 배열로 변환된 784개의 값은
  - 신경회로망을 통과하여 10개 중 하나의 범주category로 분류된다.
  - 이 때 숨어있는 학습층을 만드는 명령이 `keras.layers.Dense()`이다.
  - **Dense**는 학습을 위한 연결을 밀집된(**dense**) 구조 혹은 완전 연결(**Fully connected**)층으로 한다는 의미



# 인공 신경망을 학습시켜 보자 : 최적화와 에폭

- 신경망의 학습은
  - 추측을 한 뒤에 정답과 비교하여 오차가 얼마인지 확인하고
  - 이 오차를 줄이는 방법으로 연결의 강도를 조절하는 것이다.  
이를 최적화optimization라고 한다
- 훈련(fit)을 하기 전에 필요한 설정을 다음과 같이 해 보도록 하자.
- compile() 이라는 함수로 최적화 방법을 지정할 수 있다.

```
model.compile(optimizer='adam',  
               loss='sparse_categorical_crossentropy',  
               metrics=['accuracy'])
```

optimizer는  
adam(adaptive moment estimation) 최적화 기법을 사용.

현재 모델의 정답과 정답의 차이가 오차.

이 오차를 측정하는 것이 손실함수이다.

여기서는 sparse\_categorical\_crossentropy 함수를 사용

# 인공 신경망을 학습시켜 보자 : 최적화와 에폭

- 훈련 데이터 모음을 가지고 한 번 훈련을 실시하는 것을 에폭(epoch)이라 함. 이것을 반복함으로써 모델은 더 좋은 상태로 바뀌게 된다.  
여기서는 5개의 에폭을 지정하여 훈련을 5차례 실시한다.

```
▶ model.fit(train_images, train_labels, epochs=5)
```



Epoch 1/5

1875/1875 [=====] - 5s 2ms/step - loss: 3.3398 - accuracy: 0.6671

Epoch 2/5

1875/1875 [=====] - 4s 2ms/step - loss: 0.7457 - accuracy: 0.7346

Epoch 3/5

1875/1875 [=====] - 4s 2ms/step - loss: 0.6832 - accuracy: 0.7408

Epoch 4/5

1875/1875 [=====] - 4s 2ms/step - loss: 0.5736 - accuracy: 0.7950

Epoch 5/5

1875/1875 [=====] - 4s 2ms/step - loss: 0.5265 - accuracy: 0.8143

<keras.callbacks.History at 0x7fef5059a450>

6만개의 훈련 데이터를 사용하여  
한 번 훈련을 실시 : 1 에폭

에폭 단계마다  
손실값과 정확도가 출력

훈련이 진행될 수록  
손실값은 줄어듦  
예측의 정확도는 증가함

# 인공 신경망을 학습시켜 보자 : 최적화와 에폭

- 에폭이 진행되면서 매 에폭 단계마다 손실값 **loss**와 정확도 **accuracy**가 화면에 출력된다.
  - 훈련이 진행될 수록 손실값은 줄어듦과 예측의 정확도는 증가함
- 테스트 데이터를 이용하여 이 모델이 정답 레이블을 잘 맞히는지 확인
  - 테스트 데이터를 적용한 결과는 훈련용 데이터로 훈련한 최종 결과에 비해 정확도가 조금 떨어질 수 있다

```
▶ test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)  
print('\n테스트 정확도:', test_acc)
```

```
☞ 313/313 - 0s - loss: 0.5938 - accuracy: 0.8074 - 495ms/epoch - 2ms/step
```

테스트 정확도: 0.8073999881744385

# 학습된 신경망을 새 이미지에 적용

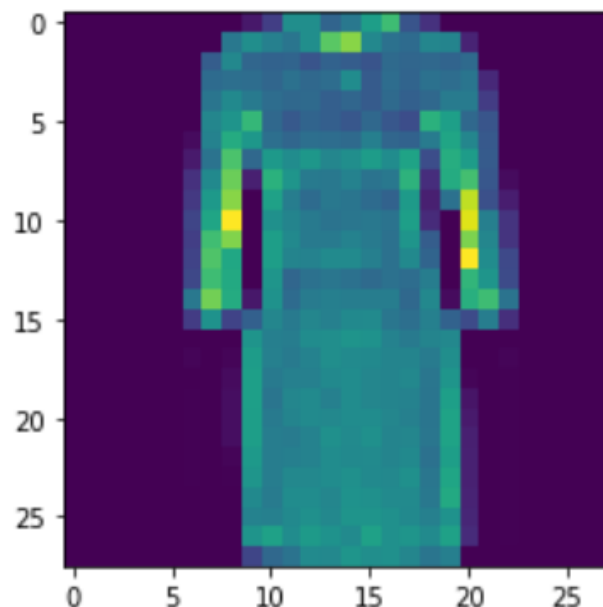
- 28x28 크기의 이미지 10,000개를 가진 3차원 배열 사이에서 하나의 데이터를 무작위로 선택해 보자. (테스트 이미지 데이터)
  - 무작위로 임의의 데이터를 선택하기 위해 넘파이의 **random**을 사용

```
test_images.shape
```

```
(10000, 28, 28)
```

```
import numpy as np  
randidx = np.random.randint(0, 1000)  
plt.imshow(test_images[randidx])
```

```
<matplotlib.image.AxesImage at 0x7f208cb09350>
```



# 학습된 신경망을 새 이미지에 적용

- 이 데이터를 모델에 집어 넣어 결과를 얻는 것은 `predict()` 함수이다.
  - `randIdx`를 `test_images`의 인덱스로 주면 **(28, 28)** 구조의 이미지가 만들어지는데, 이는 입력으로 적절하지 않다.
  - 입력에 알맞은 **(1, 28, 28)** 구조의 **shape**으로 변경시킴.

```
[ ] #yhat = model.predict(test_images[randIdx])
```

```
▶ yhat = model.predict( test_images[randIdx][np.newaxis, :, :])  
yhat
```

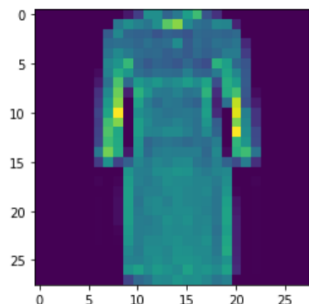
```
↪ array([[6.5727010e-02, 2.5209570e-02, 6.4698566e-04, 8.7912422e-01,  
         1.1675651e-02, 4.8779617e-07, 1.3943042e-02, 1.7364184e-13,  
         3.6728801e-03, 4.2713957e-09]], dtype=float32)
```

# 학습된 신경망을 새 이미지에 적용

- **yhat**의 예측값은 0에서 9까지 **target** 값의 확률값으로 표시된다.
  - 이 값을 살펴보면 출력 노드 10개 중에서 가장 큰 출력을 갖는 노드는 3번 노드.
  - 3번은 드레스를 의미.

```
▶ yhat = model.predict( test_images[randIdx][np.newaxis, :, :])  
yhat
```

```
↳ array([[6.5727010e-02, 2.5209570e-02, 6.4698566e-04, 8.7912422e-01,  
          1.1675651e-02, 4.8779617e-07, 1.3943042e-02, 1.7364184e-13,  
          3.6728801e-03, 4.2713957e-09]], dtype=float32)
```



# 클래스를 찾아 출력하게 만들기

- 최종 출력을 통해 가장 일치도가 높은 클래스를 찾아내기 위해 넘파이의 **argmax()** 함수를 쓸 수 있다.

```
▶ yhat = np.argmax( model.predict( test_images[randIdx][np.newaxis, :, :]) )  
yhat
```

↪ 3

- 클래스 이름을 리스트로 만든다. 리스트로 만들어 놓으면 아래와 같이 예측 결과를 쉽게 알아볼 수 있다.

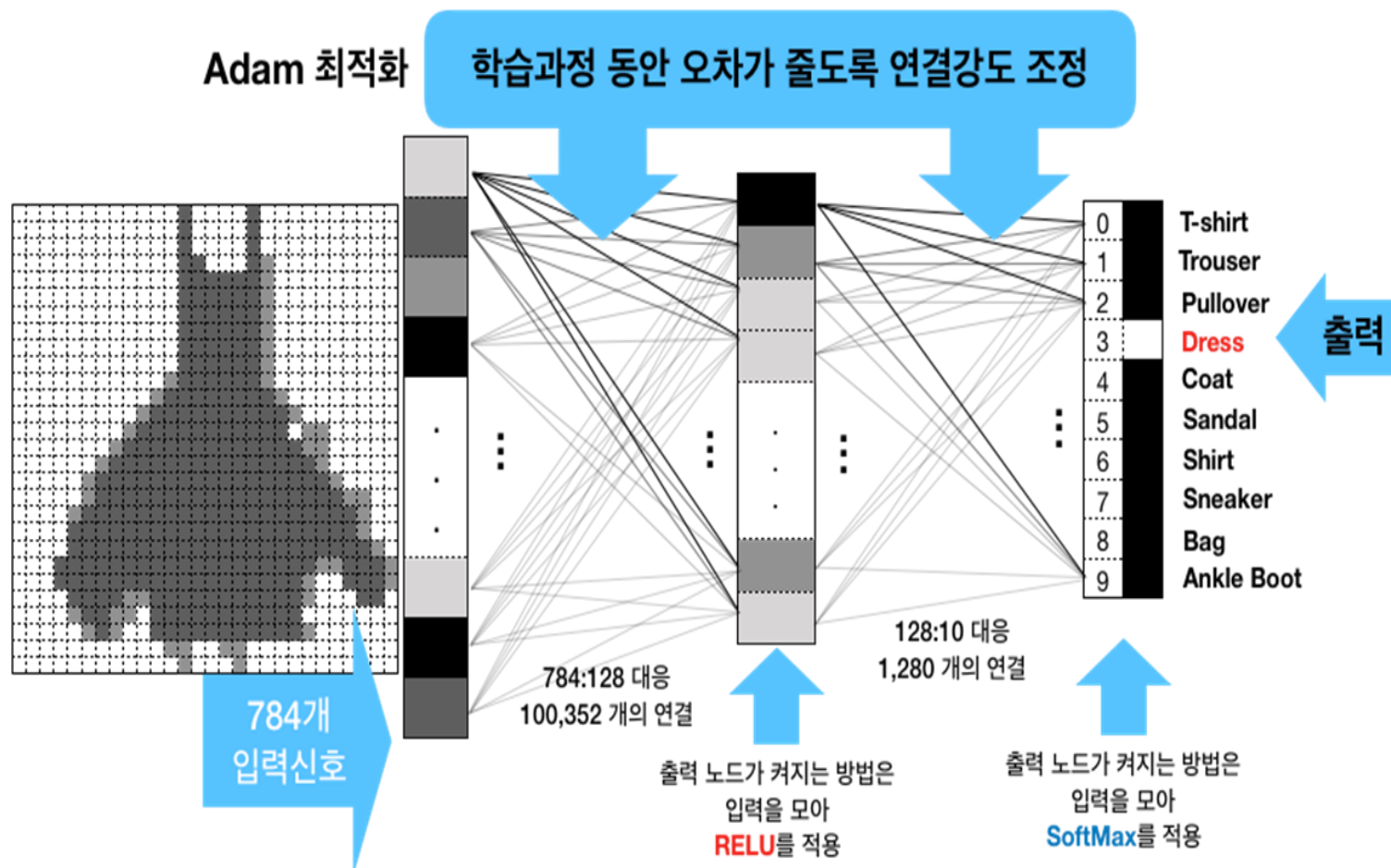
```
▶ class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
                 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

```
▶ yhat = np.argmax( model.predict( test_images[randIdx][np.newaxis, :, :]) )  
print(class_names[yhat])
```

↪ Dress

# 클래스를 찾아 출력하게 만들기

- 최적화 기법은 입력이 들어올 때마다 출력과 목표 값의 차이를 보고 신경망의 연결 강도를 조정한다.
- 이 이미지의 신호는 연결망을 따라 계산되어 가장 오른쪽에 있는 노드들 10개 가운데 드레스를 의미하는 3번 노드를 활성화



# 나만의 데이터 이용법 : 드라이브에 올린 파일 접근하기

- 가장 먼저 해야 할 일은  
자신의 드라이브를 코랩에서 사용할 수 있도록 마운트mount하는 것.  
마운트는 장치를 컴퓨터 시스템에서 접근할 수 있도록 등록하는 일이다.

```
▶ from google.colab import drive  
drive.mount('/content/drive')
```

↳ Mounted at /content/drive

- 드라이브를 코랩에 마운트시키려고 하면,  
이 작업에 대한 승인을 요청하는 페이지의 주소가 나타난다.

# 나만의 데이터 이용법 : 드라이브에 올린 파일 접근하기

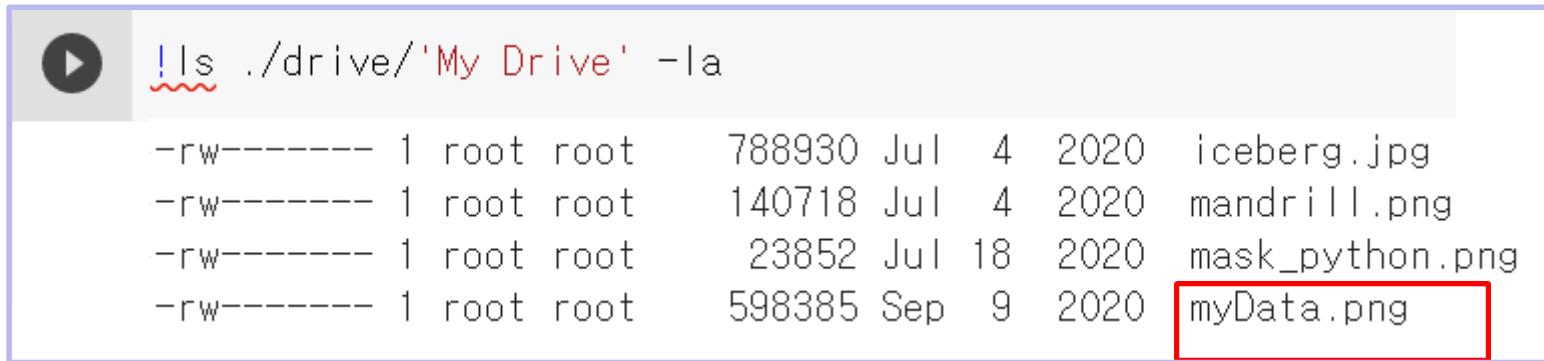
- 구글 코랩에서 현재 작업하고 있는 위치가 어디인지 알고 싶으면 `!pwd`를 입력한다.
  - `!`은 코랩의 시스템 명령을 입력하는 기호
  - `pwd`는 “print working directory”의 약어로 현재 작업중인 위치를 출력한다.

```
!pwd  
/content
```

```
!!ls ./drive/'My Drive' -la  
-rw----- 1 root root 788930 Jul 4 2020 iceberg.jpg  
-rw----- 1 root root 140718 Jul 4 2020 mandrill.png  
-rw----- 1 root root 23852 Jul 18 2020 mask_python.png  
-rw----- 1 root root 598385 Sep 9 2020 myData.png
```

# 구글 클라우드의 자료를 코랩에서 사용하기

- 구글은 구글 드라이브(클라우드 저장소)를 제공.
  - 이 저장소에 있는 콘텐츠를 코랩에서 불러서 이용할 수 있다.

A terminal window with a play button icon on the left. The command `!!ls ./drive/'My Drive' -la` is entered. The output lists four files: `iceberg.jpg`, `mandrill.png`, `mask_python.png`, and `myData.png`. The `myData.png` entry is highlighted with a red rectangular box.

```
!!ls ./drive/'My Drive' -la
-rw----- 1 root root 788930 Jul 4 2020 iceberg.jpg
-rw----- 1 root root 140718 Jul 4 2020 mandrill.png
-rw----- 1 root root 23852 Jul 18 2020 mask_python.png
-rw----- 1 root root 598385 Sep 9 2020 myData.png
```

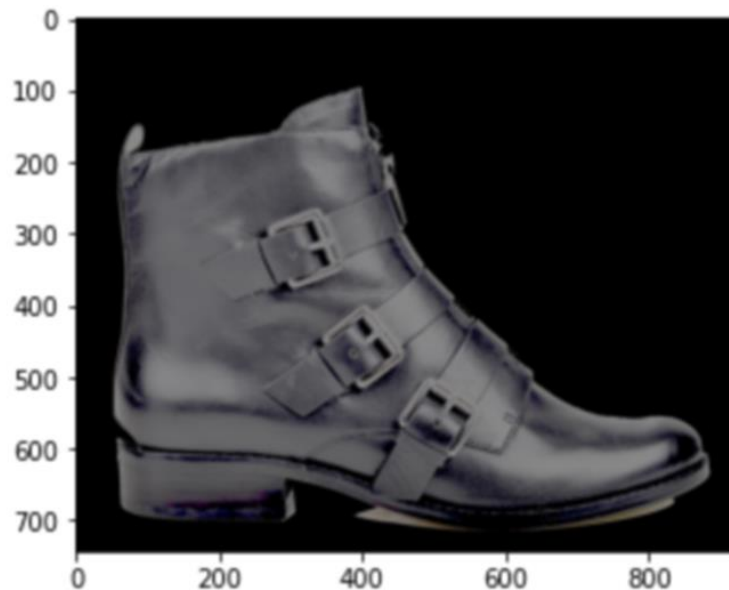
이 데이터를 업로드하고  
코랩에서 확인해 보자

# 드라이브의 이미지 파일을 읽어 화면에 표시해 보기

- 맷플롯립을 이용하여 읽어 보자.

```
▶ import matplotlib.image as mpimg  
import matplotlib.pyplot as plt  
  
img = mpimg.imread('./drive/My Drive/myData.png')  
plt.imshow(img)
```

☞ <matplotlib.image.AxesImage at 0x7f209d685d90>



# 드라이브의 이미지 파일을 읽어 화면에 표시해 보기

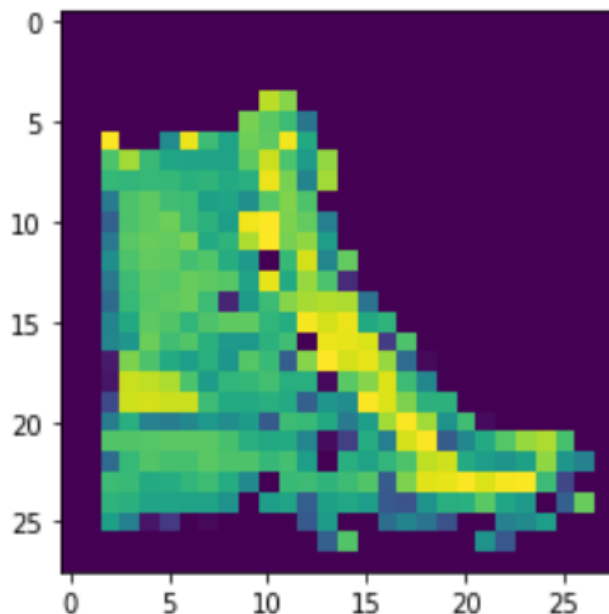
- OpenCV를 사용한다.
  - 이미지가 비로소 단일 채널이 된 것이다.  
이제 이것을 모델에 적용하면 된다.



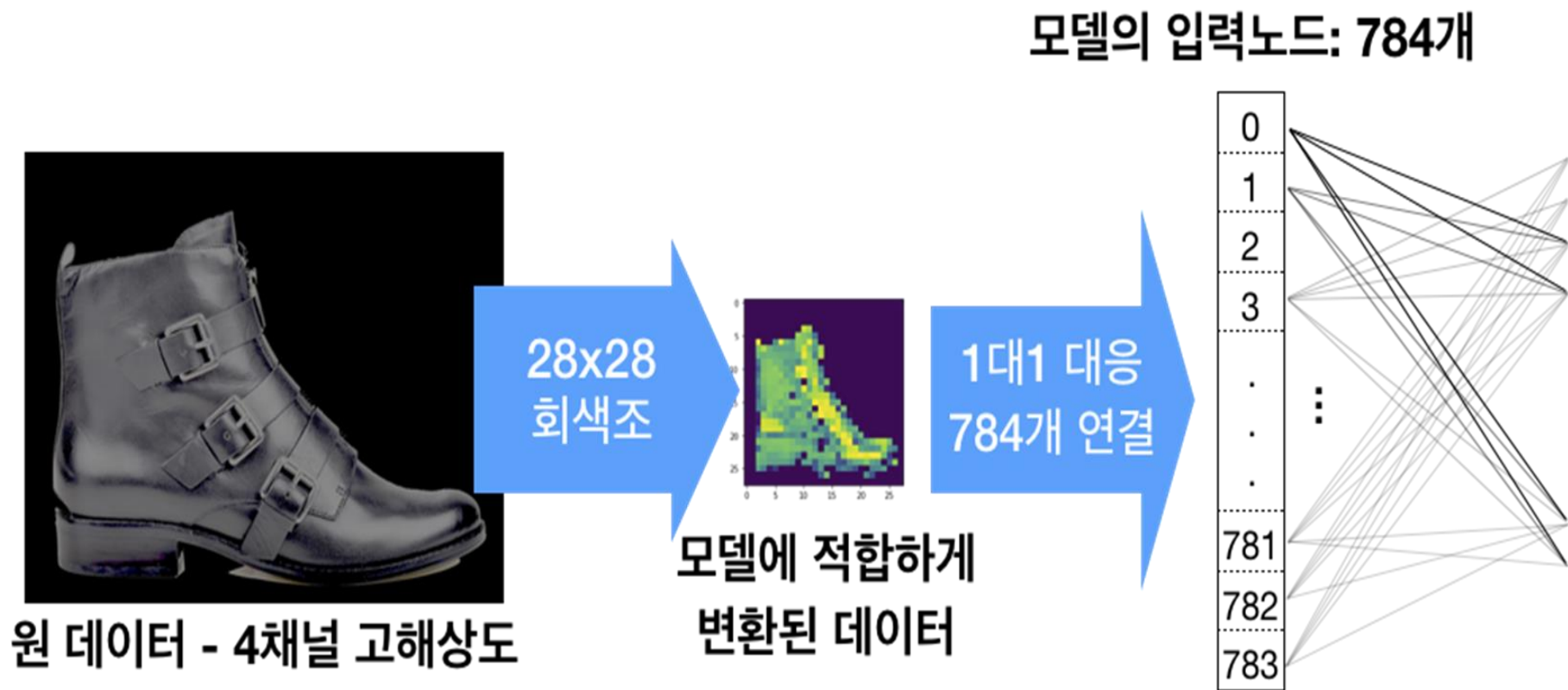
```
import cv2  
img = cv2.imread('./drive/My Drive/myData.png', cv2.IMREAD_GRAYSCALE)  
img = cv2.resize(img, (28, 28) )  
plt.imshow(img)
```



<matplotlib.image.AxesImage at 0x7f2099774790>



# 드라이브의 이미지 파일을 읽어 화면에 표시해 보기



# 이미지를 모델에 입력으로 제공해 인식을 시도하자

- 먼저 데이터를 입력 가능한 형태로 바꾸자.
  - $(n, 28, 28)$  형태의 3차원 배열이어야 하는데,
  - 현재 가지고 있는 이미지는  $(28, 28)$ 이다.
  - 이를 넘파이의 **newaxis**를 이용하여 3차원으로 바꾸자.



```
input_data = img[np.newaxis, :, :]  
input_data.shape
```



```
(1, 28, 28)
```

# 이미지를 모델에 입력으로 제공해 인식을 시도하자

- 이제 모델에게 무엇인지 맞춰보라 하자.
  - 이처럼 학습된 모델이 잘못된 결과를 출력할 수도 있다.



```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']  
yhat = np.argmax( model.predict( input_data ) )  
  
print(class_names[yhat])
```



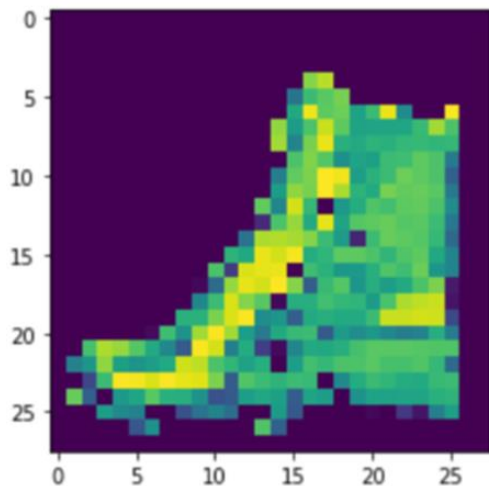
Bag

# 이미지를 모델에 입력으로 제공해 인식을 시도하자

- 좌우를 뒤집은 이미지를 다시 모델에 입력해 보자

```
▶ input_mirror = input_data[:, :, ::-1]  
plt.imshow(input_mirror[0])
```

↗ <matplotlib.image.AxesImage at 0x7f2090c7ac10>

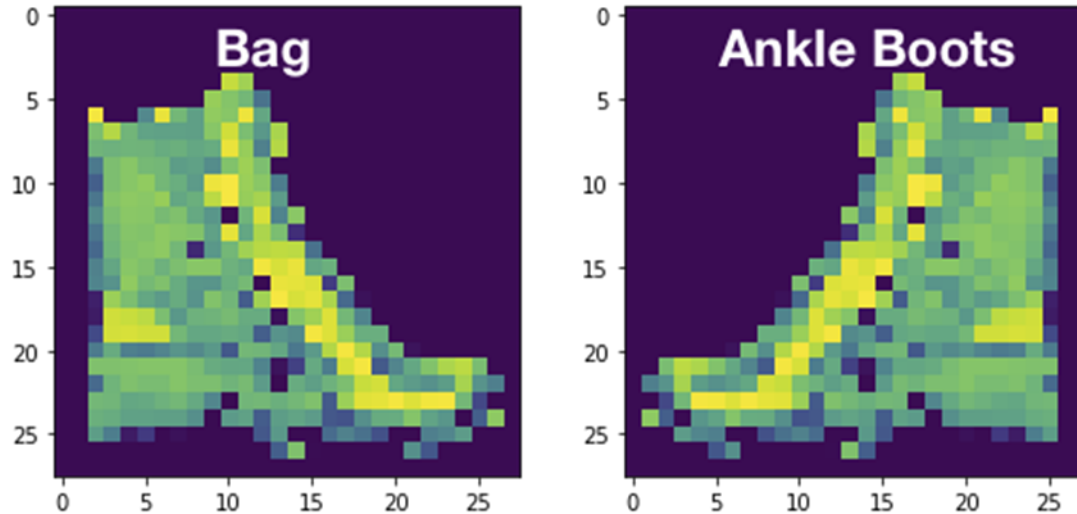


```
▶ class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',  
                 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']  
yhat = np.argmax( model.predict( input_mirror ) )  
  
print(class_names[yhat])
```

↗ Ankle boot

# 신경망 학습과 편향 : 학습의 한계를 인식하자

- 전체를 대표하지 못하고 특정한 특징이나 경향을 가진 데이터만 지나치게 학습에 많이 사용되는 것을 데이터 편향(data bias)이라 함.



- 확보된 데이터를 다양하게 변형하여 편향을 제거하는 방법이 있다. 이것을 데이터 증강(data augmentation)이라 한다.

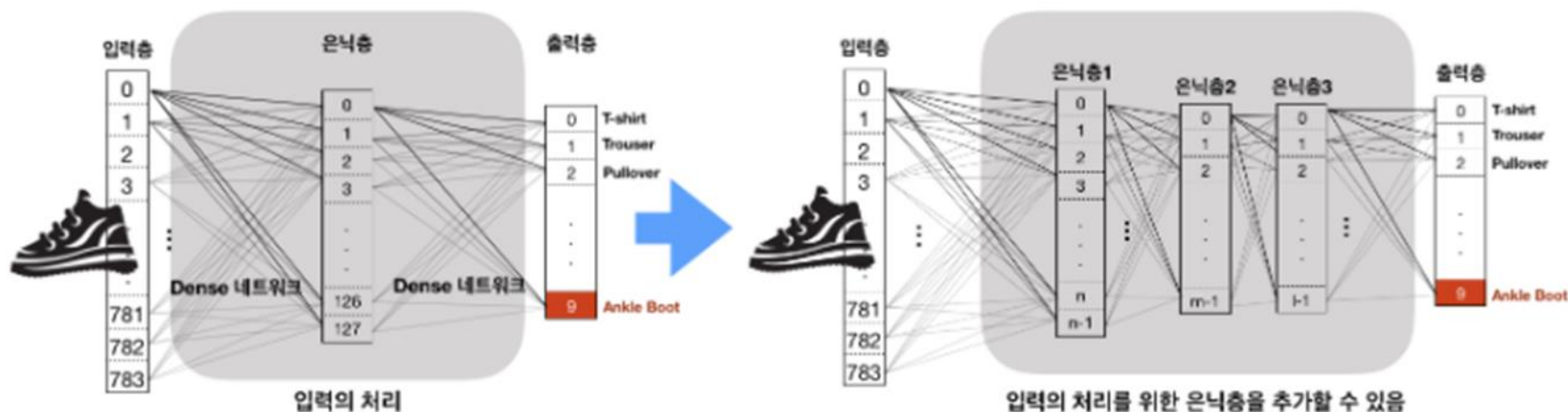
# 인공지능? 딥러닝이라는 것은 무엇인가

---

- 현재의 인공지능 열풍은 딥러닝 **deep learning**의 성공에 힘입은 바가 크다. 그래서 현재는 인공지능을 적용한다고 하면 거의 딥러닝을 적용한다는 말처럼 사용되고 있다.
- 딥러닝은 인공 신경망을 활용한 기계 학습의 일종으로 인공 신경망을 깊이 구축하는 것이다.

# 인공지능? 딥러닝이라는 것은 무엇인가

- 첫 번째 **Dense** 네트워크가 중간 은닉층(hidden layer)으로 연결이 된다. 그리고 이 은닉층의 신호가 최종적으로 물체를 분류하는 출력층에 연결되도록 다시 **Dense** 네트워크를 적용하였다.



- 이렇게 층을 쌓아 나가면 은닉층이 여러 개인 신경망을 얻을 수 있고, 이것을 심층 신경망(**deep neural network**)이라고 한다. 심층 신경망에서 학습을 진행하는 것이 딥러닝의 기본적인 개념이다.

# 인공지능? 딥러닝이라는 것은 무엇인가

```
▶ model2 = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
model2.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])

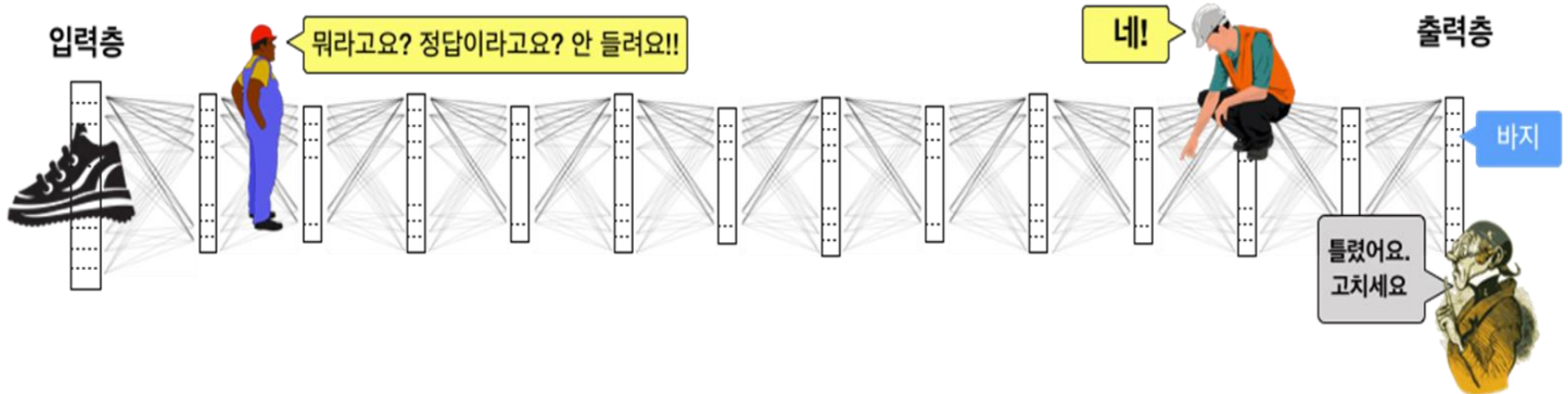
model2.fit(train_images, train_labels, epochs=5)
```

인간의 신경세포는 많은 층을 가지고 있다.  
이와 같이 모델의 층을 추가하여 보자.  
성능이 개선될까?

```
↳ Epoch 1/5
1875/1875 [=====] - 5s 3ms/step - loss: 1.3543 - accuracy: 0.6613
Epoch 2/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.6231 - accuracy: 0.7732
Epoch 3/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.5189 - accuracy: 0.8136
Epoch 4/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.4636 - accuracy: 0.8333
Epoch 5/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.4224 - accuracy: 0.8474
<keras.callbacks.History at 0x7f2090c5b8d0>
```

# 은닉층을 쌓기만 하면 좋은 모델이 될까

- 층만 깊이 쌓을 경우 출력 부분의 잘못에 근거한 학습의 내용이 입력에 가까운 네트워크 층까지 전파가 잘 되지 않는다.



- 이러한 모델이 어떻게 동작하고, 왜 좋은 결과를 내는지를 이해하는 것은 이 분야 이론에 대한 체계적이고 전문적인 학습이 필요하다.

# 학습된 모델 저장하고 불러와서 사용하기

- 모델을 저장하는 것은 매우 간단하다. 다음과 같이 **save()** 함수를 사용하면 된다.

```
▶ model2.save('./drive/My Drive/myFirstModel.h5')  
!ls ./drive/'My Drive' -la
```

|   |          |   |      |      |        |     |    |      |                 |
|---|----------|---|------|------|--------|-----|----|------|-----------------|
| ➡ | -rw----- | 1 | root | root | 788930 | Jul | 4  | 2020 | iceberg.jpg     |
|   | -rw----- | 1 | root | root | 140718 | Jul | 4  | 2020 | mandrill.png    |
|   | -rw----- | 1 | root | root | 23852  | Jul | 18 | 2020 | mask_python.png |
|   | -rw----- | 1 | root | root | 598385 | Sep | 9  | 2020 | myData.png      |

# 학습된 모델 저장하고 불러와서 사용하기

- 저장한 모델을 읽고 모델의 요약 정보를 보고 싶을 때는 다음과 같이 할 수 있다.

```
▶ model_imported = keras.models.load_model('./drive/My Drive/myFirstModel.h5')
model_imported.summary()
```

↳ Model: "sequential\_1"

| Layer (type)        | Output Shape | Param # |
|---------------------|--------------|---------|
| flatten_1 (Flatten) | (None, 784)  | 0       |
| dense_2 (Dense)     | (None, 128)  | 100480  |
| dense_3 (Dense)     | (None, 64)   | 8256    |
| dense_4 (Dense)     | (None, 32)   | 2080    |
| dense_5 (Dense)     | (None, 10)   | 330     |

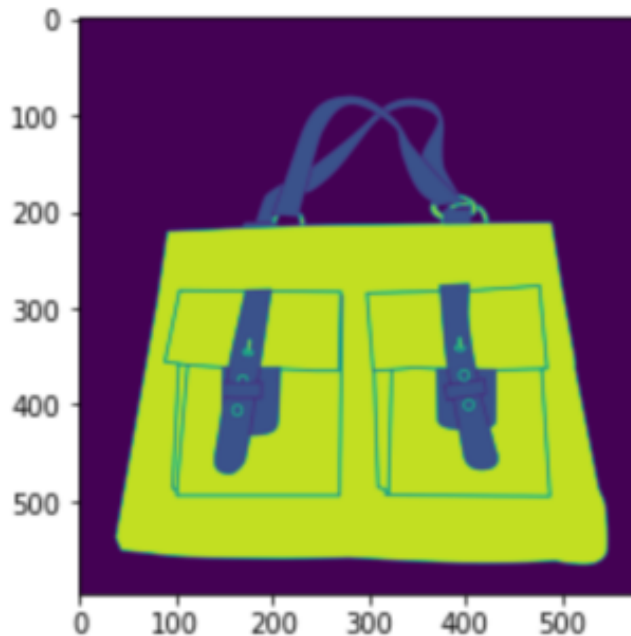
=====  
Total params: 111,146  
Trainable params: 111,146  
Non-trainable params: 0  
=====

# 학습된 모델 저장하고 불러와서 사용하기

- 실제로 이 모델을 사용하기 위해 이미지 데이터를 읽어 보자.

```
import cv2  
img = cv2.imread('./drive/My Drive/bag_cartoon.png', cv2.IMREAD_GRAYSCALE)  
plt.imshow(img)
```

 <matplotlib.image.AxesImage at 0x7f2090acacd0>

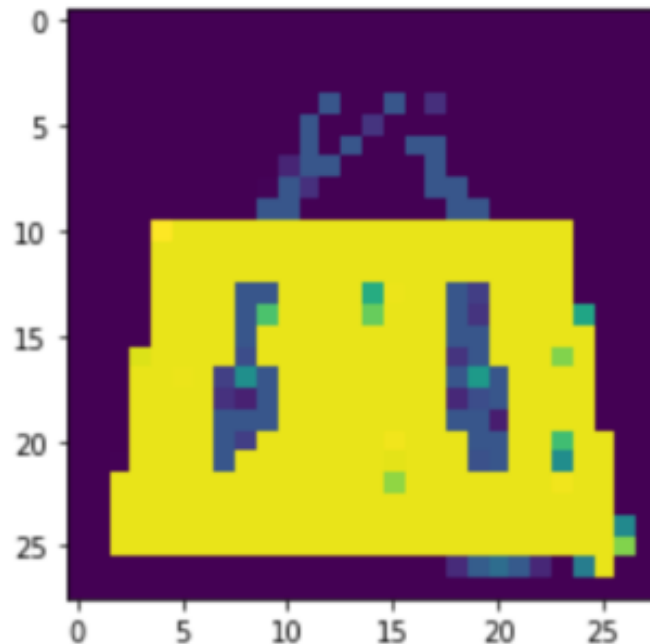


# 학습된 모델 저장하고 불러와서 사용하기

- 이 이미지를 우리가 사용하는 모델에 적용가능하도록 28x28 크기의 이미지로 바꾼다.

```
img = cv2.resize(img, (28, 28) )  
plt.imshow(img)
```

<matplotlib.image.AxesImage at 0x7f2090aa5f10>



# 학습된 모델 저장하고 불러와서 사용하기

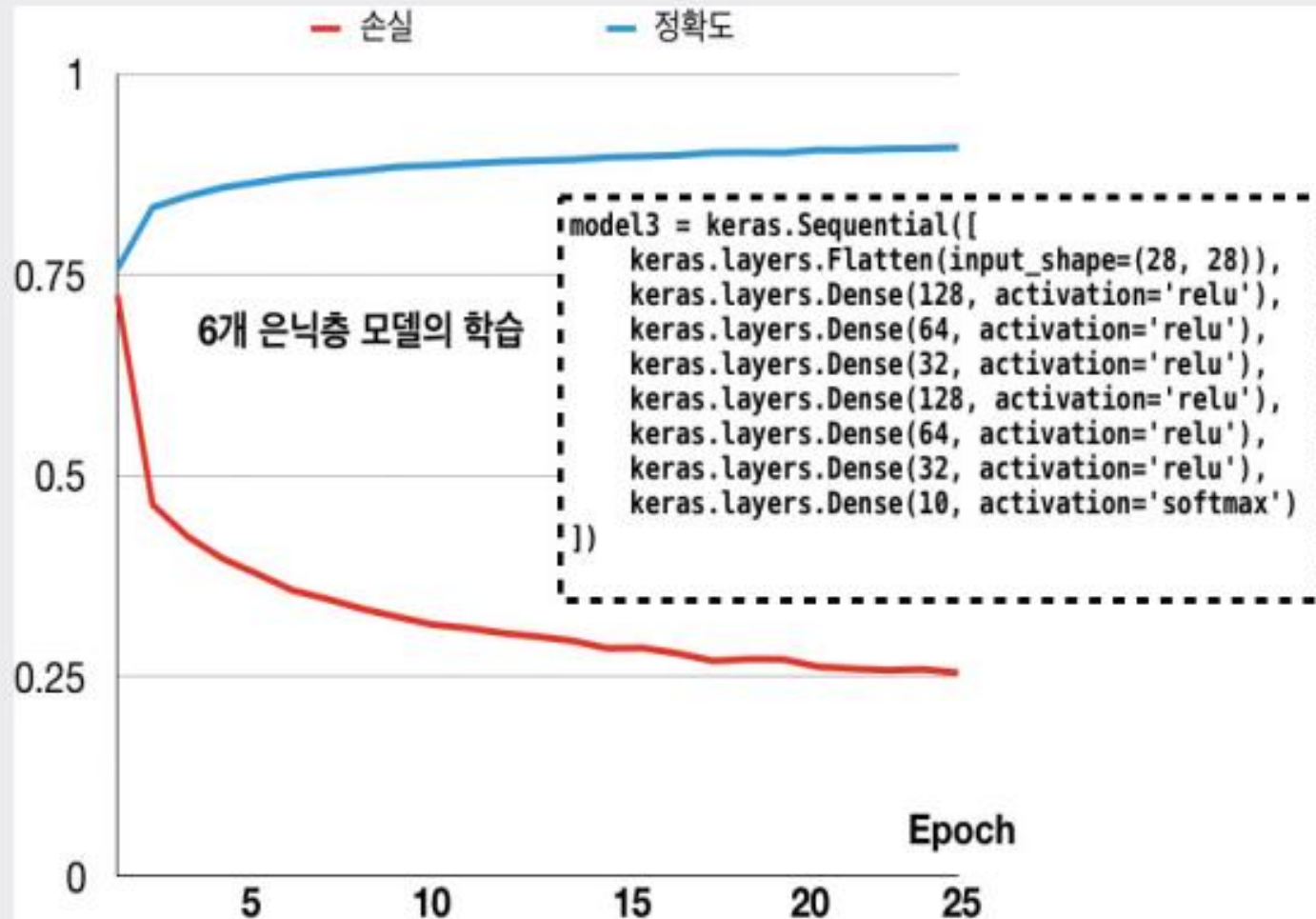
- 이 그림이 무엇을 나타내는지 인식해 보자.

```
▶ input_data = img[np.newaxis, :, :]  
yhat = np.argmax( model_imported.predict( input_data ) )  
print(class_names[yhat])
```

📄 Bag

- 이제 우리는 모델을 학습시키고,  
학습된 결과를 저장할 수 있으며,  
필요할 때 이 모델을 가지고 와서 사용할 수 있게 되었다.

# 학습된 모델 저장하고 불러와서 사용하기



# 학습된 모델 저장하고 불러와서 사용하기

---

- model2는 은닉층이 6개가 되는 모델.
  - 훈련에 앞서 Fashion\_MNIST 데이터를 사용
  - 25 에폭에 걸쳐서 학습 시킴
  - 빨간 선은 학습 후 손실의 변화.
  - 파란 선은 정확도의 변화를 보임
- 학습 정도를 관찰한 결과
  - 시간이 지남에 따라 정확도가 높아지기는 하지만, 금세 정체에 도달
  - 따라서, 이 모델을 아무리 오랫동안 학습시킨다 하더라도 오차가 거의 없는 수준의 분류기로 발전시키기는 어렵다
  - 좋은 분류기는 단순히 층을 쌓아 올리는 것 이상의 방법이 필요함